

Simulación Numérica de la Propagación de Grietas Dinámicas en 2D mediante el Método de Diferencias Finitas en Python

Andrés Villava Sergio, Buiza Reyes Yoel, Cárdenas Chilon Jean
Facultad de Ingeniería Mecánica, Universidad Nacional del Callao, Perú

Mayo 2026

Abstract

En este trabajo se presenta una simulación computacional de la propagación dinámica de grietas en una placa elástica bidimensional utilizando el Método de Diferencias Finitas (MDF). El modelo resuelve la ecuación de onda mediante un esquema explícito centrado. La fractura se modela como una discontinuidad de malla que anula la interacción entre nodos vecinos. El algoritmo, implementado en Python, utiliza un criterio de propagación basado en el gradiente de desplazamiento local. Los resultados demuestran la capacidad del MDF para capturar la concentración de esfuerzos en la punta de la grieta y la interacción compleja con ondas mecánicas incidentes.

1. INTRODUCCIÓN

El análisis de la integridad estructural requiere modelos precisos para predecir el comportamiento de fallas catastróficas. La mecánica de fractura computacional ha evolucionado desde métodos analíticos hacia simulaciones numéricas robustas. En este estudio, se emplea el MDF debido a su eficiencia para resolver problemas dinámicos transitorios, permitiendo visualizar en tiempo real la evolución de una grieta bajo perturbaciones elásticas.

2. FORMULACIÓN MATEMÁTICA

La propagación de ondas en un medio elástico lineal e isótropo se rige por la ecuación de onda:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \nabla^2 u \quad (1)$$

Donde $u(x, y, t)$ es el desplazamiento escalar y $c = \sqrt{E/\rho}$ es la velocidad de la onda. Aplicando diferencias finitas centradas, la actualización temporal se define como:

$$u_{i,j}^{n+1} = 2u_{i,j}^n - u_{i,j}^{n-1} + r^2 [u_{i+1,j}^n + u_{i-1,j}^n + u_{i,j+1}^n + u_{i,j-1}^n - 4u_{i,j}^n] \quad (2)$$

donde $r = c \frac{\Delta t}{\Delta x}$ es el número de Courant.

3. IMPLEMENTACIÓN COMPUTACIONAL

El script `ejerfin.py` estructura la simulación en ciclos iterativos. Se define una grilla de 100×100 nodos con una condición inicial de perturbación gaussiana:

$$u(x, y, 0) = e^{-100((x-0.5)^2 + (y-0.5)^2)} \quad (3)$$

3.1. Lógica de la Fractura

La grieta se gestiona mediante un conjunto de coordenadas (`crack_set`). En cada paso de tiempo, se evalúa si el gradiente en la punta de la grieta supera el esfuerzo crítico σ_c :

$$\sigma = E \cdot \sqrt{\left(\frac{\Delta u}{\Delta x}\right)^2 + \left(\frac{\Delta u}{\Delta y}\right)^2} \quad (4)$$

4. RESULTADOS

La simulación revela que la grieta dispersa las ondas mecánicas generando patrones de interferencia. La velocidad de propagación de la punta de la grieta se mantiene inferior a c , cumpliendo con los límites teóricos de la mecánica de fractura dinámica.

5. CONCLUSIONES

El uso de Python y el MDF ofrece una plataforma didáctica y potente para el estudio de fracturas. La precisión del modelo depende directamente del refinamiento de la malla y la correcta calibración del esfuerzo crítico.

References

- [1] Anderson, T. L. (2017). *Fracture Mechanics: Fundamentals and Applications*. CRC Press.
- [2] Smith, G. D. (1985). *Numerical Solution of Partial Differential Equations*. Oxford University Press.

A. MÓDULO DE ACTUALIZACIÓN EN PYTHON

```
1 for i in range(1, Nx-1):
2     for j in range(1, Ny-1):
3         if not es_crack(i,j):
4             u_next[i,j] = (2*u[i,j] - u_prev[i,j] + r
                             **2 * laplacian(u, i, j))
```